

Store same-type values

```
from array import array
nums = array('i',
             [10, 20, 30])
```

Arrays store many values of one type.

Useful when type consistency and memory matter.

Lists are flexible

List

```
values = [10, "Satya", 4.5]  
# mixed types allowed  
# stores object refs  
# flexible, but heavier
```

Array

```
nums = array('i', [10, 20])  
# same type only  
# compact values  
# good for typed data
```

Use arrays when every element should share one type.

Import and type code



Setup

```
from array import array
nums = array('i', [5, 10, 15])
# 'i' = signed integer
```

Import

array lives in Python's built-in array module.

Type code

Tell Python what kind of values the array stores.

'i'

A common type code for signed integers.

Create and inspect



Create an array

```
from array import array
vals = array('i',
             [3, 6, 9, 12])
print(vals)
```



Output

```
array('i', [3, 6, 9, 12])

type code: 'i'
values: 3, 6, 9, 12
```

Create arrays with a type code plus the values.

Loop through arrays



Iteration

```
from array import array
nums = array('i', [2, 4, 6])
for n in nums:
    print(n)
```

for loop

Process each value one by one.

Typed data

Every item follows the chosen type code.

Cleaner

No manual index needed for basic iteration.

Memory with `buffer_info()`

Inspect storage

```
nums = array('i', [2, 4, 6])  
nums.buffer_info()
```

```
# returns:  
(address, length)
```

Meaning

address
where array data starts

length
number of elements

`buffer_info()` shows storage address and item count.

Cheat Sheet

Python array module

Basic syntax

```
from array import array
nums = array('i', [1, 2, 3])
for n in nums:
    print(n)
nums.buffer_info()
```

Same type

Arrays store homogeneous values.

Type code

Use codes like 'i' for signed integers.

Efficient

Useful for compact typed collections.

NumPy next

Use NumPy for advanced or multi-dimensional arrays.

Save this for your Python journey.

Day 28/55 - Prepared by Satyanarayan Sen